

UNIT III: Bootstrap and WordPress

Bootstrap is a powerful, free, and open-source front-end framework for developing responsive, mobile-first websites and web applications. Developed by Twitter, it contains a collection of HTML, CSS, and JavaScript-based design templates for common interface components like typography, forms, buttons, navigation, modals, and more.

The primary goal of Bootstrap is to make web development **faster and easier** by providing a pre-built, consistent foundation, allowing developers to focus on the application's functionality rather than reinventing basic styling and responsiveness.

features of Bootstrap: Responsive, Mobile-First Grid System, Pre-styled Components, Browser Compatibility, Extensive Documentation, Customizable, JavaScript Plugins

key components of Bootstrap

1. **Layout System: Grid System**
2. **CSS Content & Components:** This includes pre-styled, ready-to-use interface elements.
 - **Content:** Global styles for typography, code blocks, images, and tables.
 - **Components:** Reusable UI elements like **Buttons, Forms, Navbars, Cards, Modals, Alerts, and Carousels.**
3. **JavaScript Components:** These are interactive components that require JavaScript to function
4. **Utility Classes:** These are helper classes that allow you to quickly apply common styling properties without writing custom CSS. They cover things like **spacing** (m-1, p-2), **colors** (text-primary, bg-success), **display** (d-flex), **flexbox alignment**, and **text alignment**.

Bootstrap Grid System is a powerful, mobile-first system for building responsive layouts. It uses a series of containers, rows, and columns to align and structure content.

How it Works: 12-Column Structure, Rows and Columns, Responsive Breakpoints: The system uses predefined breakpoints (e.g., sm for small, md for medium, lg for large screens) to control how the layout changes with screen size.

Bootstrap Navbar is a responsive and powerful navigation header component that is placed at the top of a webpage. It is one of the most commonly used components for site branding and navigation.

Key Features:

- **Branding:** Provides a dedicated spot for a site logo or name using the .navbar-brand class.
- **Navigation Links:** Supports lists of navigation links using the .navbar-nav class.
- **Responsive Collapse:** On smaller screens, the navigation links can be collapsed into a "hamburger" menu, which is toggled using the .navbar-toggler button.
- **Color Schemes:** Easily styled with color schemes like .navbar-light (for light backgrounds) and .navbar-dark (for dark backgrounds) along with background color utilities (.bg-primary, .bg-light, etc.).
- **Content Support:** Can include other elements like forms, text, and dropdown menus.

Class	.container	.container-fluid
Width	Responsive fixed-width. Its max-width changes at each responsive breakpoint to adapt to the screen size.	Full-width. It always spans 100% of the width of the viewport.
Appearance	On larger screens (desktops), it appears centered with empty space (margins) on the left and right.	It always fills the entire horizontal space of the screen, with no side margins.
Common Use Case	Standard websites where content should be centered and have a readable line length (e.g., blogs, corporate sites).	Applications or websites where you need an edge-to-edge layout (e.g., dashboards, image-heavy landing pages).

Bootstrap's responsive utilities are a set of helper classes that allow you to **show or hide** elements based on the current screen size or viewport. They are a powerful tool for creating tailored experiences across different devices without writing custom media queries.

How they work: These utilities apply CSS display properties based on Bootstrap's responsive breakpoints (xs, sm, md, lg, xl, xxl). The format is typically .d-{breakpoint}-{display}.

How to use them: You simply add the appropriate class to an HTML element.

- **Hiding elements:** To hide an element on certain screens, you use `.d-{breakpoint}-none`.
 - `d-none`: Hides the element on all screen sizes.
 - `d-md-none`: Hides the element on medium screens and larger.
- **Showing elements:** To show an element only on certain screens, you combine hiding and showing classes.
 - `.d-none .d-lg-block`: This combination will hide the element by default (`.d-none`) and then make it visible as a block-level element only on large screens and up (`.d-lg-block`).

Example: Display a different message on mobile vs. desktop.

```
<div class="d-md-none">
  <p>This is the mobile view.</p>
</div>
<div class="d-none d-md-block">
  <p>This is the desktop view.</p>
</div>
```

WordPress is a free, open-source **Content Management System (CMS)**. It is written in PHP and uses a MySQL or MariaDB database to store its content. While it originally started as a platform for blogging, WordPress has evolved into a versatile and powerful tool used to create almost any type of website, including: Blogs and personal websites, Corporate and business websites, E-commerce stores (using plugins like WooCommerce), Portfolios, Forums and membership sites

features of WordPress: User-Friendly Interface, **Extensibility** with Themes and Plugins, **Search Engine Optimization (SEO)** Friendly, **Open Source** and Community Driven, User Management, Media Management

Feature	Posts	Pages
Purpose	Timely content. They are blog entries or articles that are displayed in reverse chronological order.	Static, "timeless" content. They are not tied to a specific date.
Organization	Organized using Categories and Tags .	Hierarchical. They can be organized into a parent/child structure.
Display	Appear on the blog or archive pages of a site.	Typically accessed through the site's main navigation menu.
RSS Feed	Included in the site's RSS feed to notify subscribers of new content.	Not included in the RSS feed.
Examples	A news article, a company announcement, a weekly blog update.	"About Us," "Contact," "Privacy Policy," "Services."

In **WordPress, widgets** are small, independent blocks of content that can be placed in specific, predefined areas of a theme. These areas are known as **widget areas** or **sidebars** (though they can be in the footer, header, or anywhere the theme developer decides). Widgets allow users to add content and features to their site in a simple, drag-and-drop manner, without needing to write any code.

improve the performance of a WordPress site

1. **Use Caching:** Install a caching plugin (e.g., W3 Total Cache, WP Rocket). Caching creates static HTML copies of your pages, so they don't have to be generated from the database for every single visitor, dramatically reducing server load and speeding up delivery.
2. **Optimize Images:** Large image files are a common cause of slow sites. Compress images before uploading them, or use a plugin like Smush to do it automatically. Also, use properly sized images for their containers.
3. **Choose a High-Quality Hosting Provider:** Cheap, shared hosting can be slow. A good hosting provider optimized for WordPress can make a significant difference in site speed.
4. **Use a Content Delivery Network (CDN):** A CDN stores copies of your site's static assets (images, CSS, JS) on servers around the world. It serves these assets to visitors from the server closest to them, which reduces latency and speeds up load times.
5. **Keep Everything Updated:** Regularly update the WordPress core, your themes, and all plugins. Updates often include performance improvements and security patches.
6. **Optimize the Database:** Use a plugin like WP-Optimize to clean up your WordPress database by removing old post revisions, spam comments, and other unnecessary data.

7. **Use a Lightweight Theme:** Avoid themes that are bloated with too many features you don't need. A well-coded, lightweight theme provides a faster foundation.

handle database management in WordPress

WordPress has a built-in set of functions and a dedicated class for interacting with its database safely and efficiently. Direct database management is typically done in one of two ways:

For Developers (The Correct Way): Developers should never use standard PHP MySQL functions (like `mysqli_query`). Instead, they must use the global `$wpdb` object, which is an instance of the `wpdb` class. This object provides a secure way to interact with the database.

- **Key \$wpdb Methods:**
 - `$wpdb->get_results()`: To retrieve multiple rows from the database.
 - `$wpdb->get_row()`: To get a single row.
 - `$wpdb->get_var()`: To get a single value from a query.
 - `$wpdb->insert()`, `$wpdb->update()`, `$wpdb->delete()`: For safe data manipulation.
- **Security:** To prevent **SQL injection attacks**, it is absolutely critical to use `$wpdb->prepare()` to sanitize all data before it is included in a query.

```
global $wpdb;
$user_id = 123;
$status = 'active';
$results = $wpdb->get_results( $wpdb->prepare(
    "SELECT * FROM {$wpdb->prefix}users WHERE ID = %d AND status = %s",
    $user_id,
    $status
));
```

For Non-Developers (Using Plugins): Most users manage their database through plugins that handle tasks like:

- **Backups:** Plugins like UpdraftPlus can schedule and create full database backups.
- **Optimization:** Plugins like WP-Optimize can clean up the database by removing old revisions, spam comments, and other junk data to improve performance.⁷

The **WordPress REST API** provides a way for other applications (like mobile apps or JavaScript-based front-ends) to interact with a WordPress site's data programmatically over HTTP. It exposes the site's content (posts, pages, users, etc.) in a predictable JSON format.

How to Use It:

1. **Endpoints:** The API is accessed through URLs called endpoints. The base endpoint is typically `your-site.com/wp-json/`. For example, to get a list of your most recent posts, you would send a GET request to:
`https://your-site.com/wp-json/wp/v2/posts`
2. **HTTP Methods:** It uses standard HTTP methods for **CRUD** (Create, Read, Update, Delete) operations:
 - GET: To retrieve data. (e.g., `GET /wp/v2/posts/123` to get post with ID 123).
 - POST: To create new data. (e.g., `POST /wp/v2/posts` to create a new post).
 - PUT/PATCH: To update existing data.
 - DELETE: To delete data.
3. **Authentication:** For read-only actions (GET), no authentication is usually needed for public content. However, for actions that modify data (POST, PUT, DELETE), you must authenticate yourself. This can be done using application passwords, OAuth2, or cookie authentication for logged-in users.

Use Case (Headless CMS): A popular use case is a "headless" setup where WordPress serves only as the back-end for managing content, and the front-end is a separate application (e.g., built with React or Vue) that fetches and displays this content using the REST API.⁷⁴
